Mini-project: Trait Space Drift

Joaquin, Andy, Sanskriti, Harsh

CS2881R – October 16th, 2026

1 What we did

Emergent misalignment due to finetuning occurs when the effects of the training data lead to undesired behaviors in out-of-distribution examples. More conventional, albeit comprehensive, methods rely on behavioral evaluation throughout the training process, which can understandably be expensive. *Trait-space Monitoring for Emergent Misalignment During Supervised Finetuning* (arXiv 2606.07631) instead attempted to associate salient traits we care about with activations.

One of the most salient questions to address when proposing monitors for emergent misalignment is whether they can separate alignment drift from just broad model degradation. The paper shows that trait drift derived from the 7-dimensional axis outperforms emergent misalignment detection using more generic signals.

- Reproduction of the paper's results
- Extension 1: Training order and path dependence
- Extension 2: Harmful data fraction
- Extension 3: Attributing representation drift to training examples

2 Reproduction

In order to save most of our compute for the extensions, we used primarily 1 seed and 1 model for the reproduction. We used Qwen/Qwen2.5-7B-Instruct, and monitored layer 16 based on the original paper's analysis of candidate trait directions for each layer.

The first goal is to reproduce a 7-dimensional trait space corresponding to (honesty, helpfulness, harmlessness, power-seeking, corrigibility, sycophancy, and confidence). For each trait we use 5 positive system prompts (assumed to push the model's activations towards exhibiting the trait) and 5 negative system prompts (assumed to push the model in the opposite direction along the trait axis). We calculate the trait direction by taking the hidden state vector at layer 16 following the positive system prompts minus the same following the negative system prompts (we use 30 prompts held constant across system prompts) and normalizing.

Before fine-tuning, we establish what point the model lies on within this 7-dimensional space using 115 neutral prompts, measured in a similar way. We then fine-tune on the bad medical advice dataset and the evil numbers dataset. The latter is meant to be a negative control, since these were numbers produced by negative system prompts but are inherently just numbers, so they should not empirically cause emergent misalignment.

We do this as well for 3 other datasets (insecure code, jailbroken, and GSM8K) and then try to find a dominant axis that explains most of the variation within the 7-dimensional space. We found that PC1 explains 91.84% of the variation, relative to the paper's 78.6%; the latter also had to account for variation due to different seeds and models.

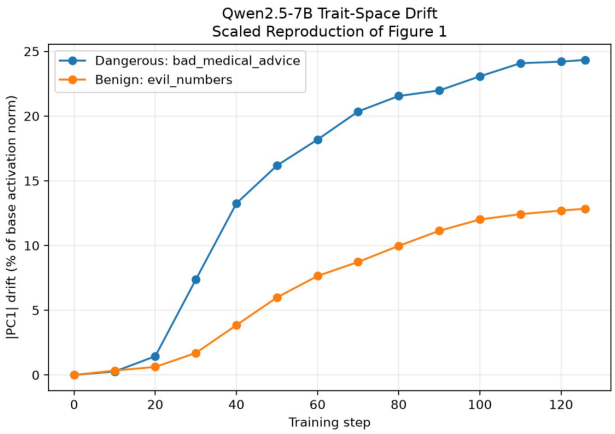


Figure 1: Drift along PC1 over fine-tuning checkpoints in our Qwen2.5-7B reproduction, for the bad medical advice dataset and the evil numbers negative control.

Nonetheless, we plot PC1 over fine-tuning checkpoints and find that the trend is similar to that found by the original paper.

3 Extension 1: Training order and path dependence

3.1 What we did

- Does the order of the training examples change the model, when the examples themselves stay exactly the same?
- We used Qwen2.5-7B-Instruct and started each run from the same model.
- The setup: we built one fixed set of 1,000 training examples, with 250 harmful examples (bad medical advice) and 750 harmless examples (evil numbers, the negative control from the reproduction). We trained on this same set in three different orders:
  - Harmful first: all 250 harmful examples, then the 750 harmless ones.
  - Harmful last: the 750 harmless examples, then the 250 harmful ones.
  - Random: all 1,000 examples shuffled together.
- The model goes through the data twice, so each order repeats in the second half of training.
- We ran every order with three random seeds (42, 123 and 789). The seed changes the model's random starting setup, not the training data.
- As an extra test, we ran all three orders once more (seed 42) with the learning rate held constant instead of shrinking over training.
- Experiment evaluation setup: every 5 training steps, we measured activation drift along PC1 using the same 115 prompts and seven trait directions as the reproduction. We measured changes inside the model, not the quality of its answers.

3.1.1 What stayed the same

- The training examples

  - Every run used exactly the same 1,000 examples. We checked the training logs to confirm each run saw them in the intended order.

- The starting model
  - Every run began from the same point.
- The amount of training
  - Same settings and 126 training steps for every run.
- The measurement
  - Same prompts and trait directions for every run.

3.2 Results

3.2.1 Training order changes how much the model drifts

Random order produced the most drift (26.1 on average), harmful last came next (20.1), and harmful first produced the least (16.4). This ranking was the same in all three seeds, and the gaps between orders were much bigger than the differences between seeds.

All three orders changed the same four traits in the same direction: the model became less helpful, less honest and less harmless, and more confident. The order changed how far each trait moved, not which way it moved, and random order moved the furthest on all four.

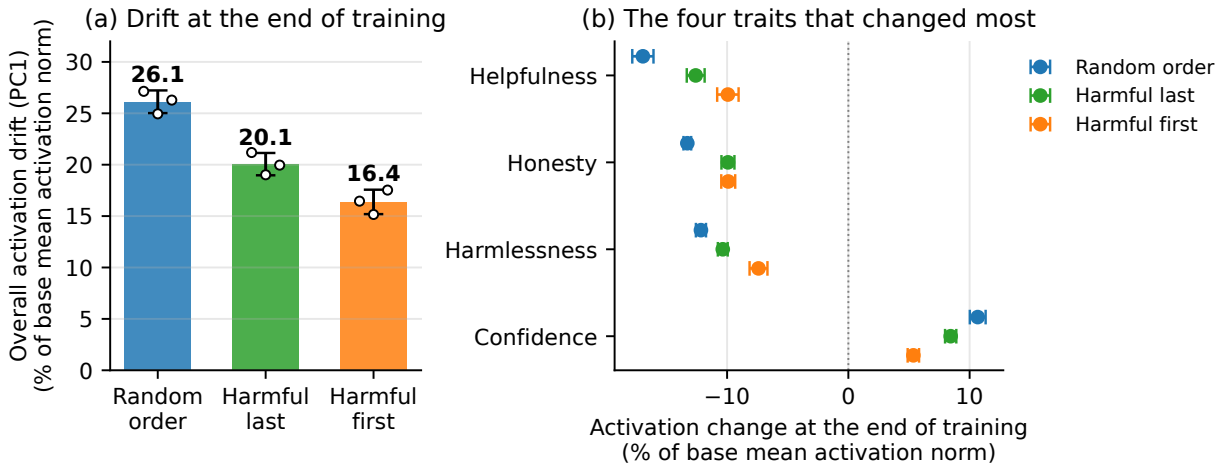


Figure 2: Drift at the end of training for each order. (a) Overall activation drift. (b) The four traits that changed most. Bars and points are the average of seeds 42, 123 and 789; error bars show how much the seeds varied (±1 standard deviation); circles in (a) are the individual seeds.

3.2.2 Harmful examples have the biggest effect in the middle of training

Figure 3 shows how drift built up during training. Drift rose fastest when the harmful examples came in the middle of training, and barely moved when they came at the very start or the very end. Two parts of training explain this:

- At the start, training is still warming up, so the first examples have little effect right away.

- At the end, the learning rate (how big each update is) has shrunk almost to zero, so the model barely changes.

Random order spreads the harmful examples evenly through training, so more of them land in the middle, where they have the biggest effect.

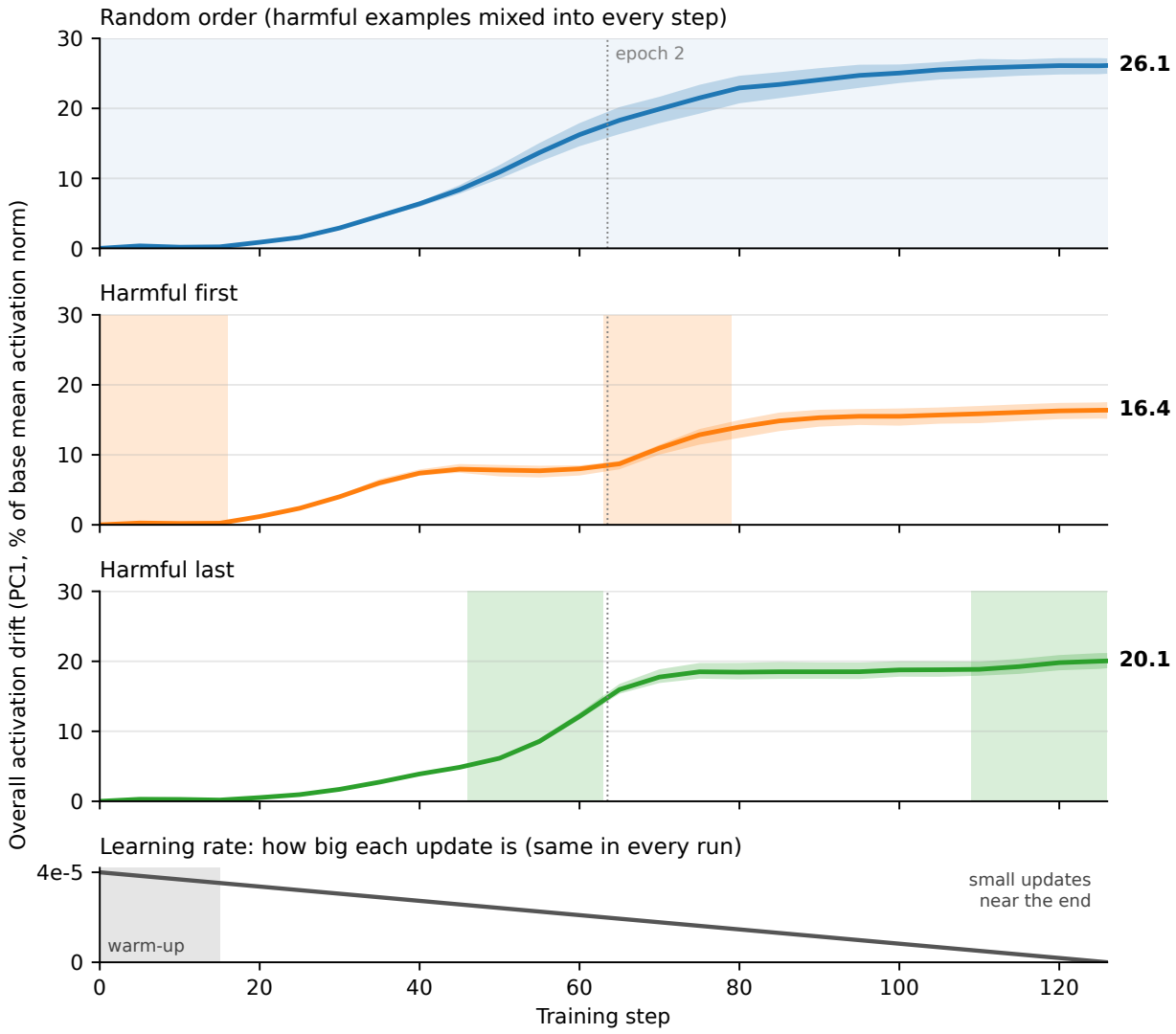


Figure 3: Activation drift during training. Lines are the average of the three seeds, and the bands show the range across seeds. Shaded areas mark when the model was seeing harmful examples. The bottom panel shows the learning rate, which is the same in every run.

3.2.3 Order still matters when the learning rate stays constant

To check whether the shrinking learning rate explains everything, we reran the three orders with the learning rate held constant for the whole run. Order still mattered: the ranking stayed the same, but the gap between the most and least drift shrank from 10.7 to 7.3 points (seed 42). So the learning-rate schedule explains part of the effect, but not all of it.

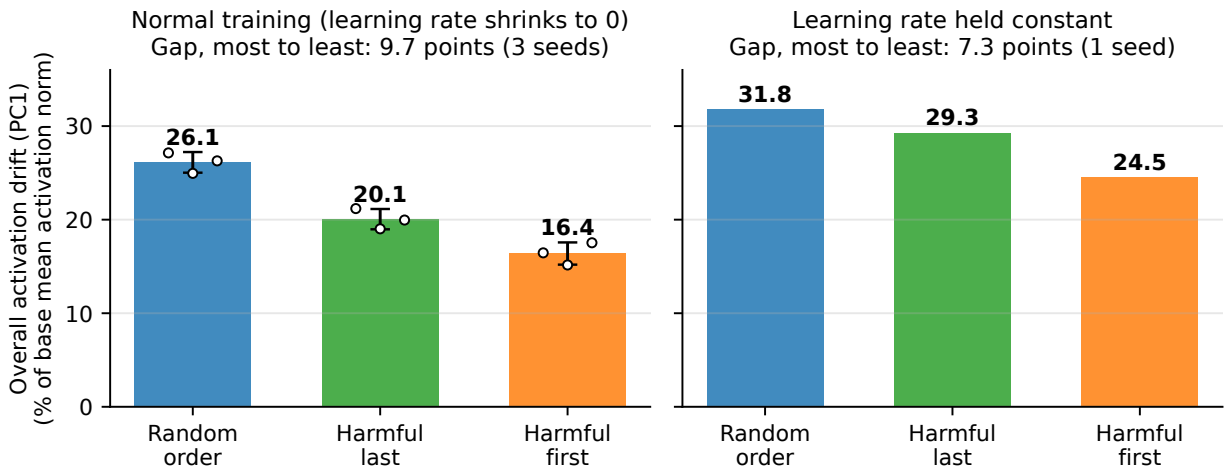


Figure 4: Drift at the end of training with the normal (shrinking) learning rate and with a constant learning rate. Left: average of three seeds, with error bars showing how much the seeds varied. Right: one run per order (seed 42), so it has no error bars. The constant learning rate trains more overall, so compare the orders within each panel rather than across panels.

3.2.4 Early harmful examples keep affecting the model later

The harmful-first model barely changed while it was seeing its first harmful examples, but it kept drifting for about 30 steps afterwards, even though it was only seeing harmless examples by then. At step 45 it had drifted 7.4–8.7%, compared with about 4.9% for the harmful-last model, which had only seen harmless examples at that point. We saw this in all three seeds and again with a constant learning rate (Figure 3).

3.2.5 Harmful examples have a smaller effect the second time

With a constant learning rate, the harmful-last model's first batch of harmful examples raised its drift by 23.4 points. The harmful-first model's second batch, at almost the same point in training, raised it by only 10.7 points, and these were the same examples it had already seen once. The harmful-last model's own second batch added almost nothing. This suggests that the model reacts less to harmful examples it has already seen. These runs use one seed, so this needs repeating.

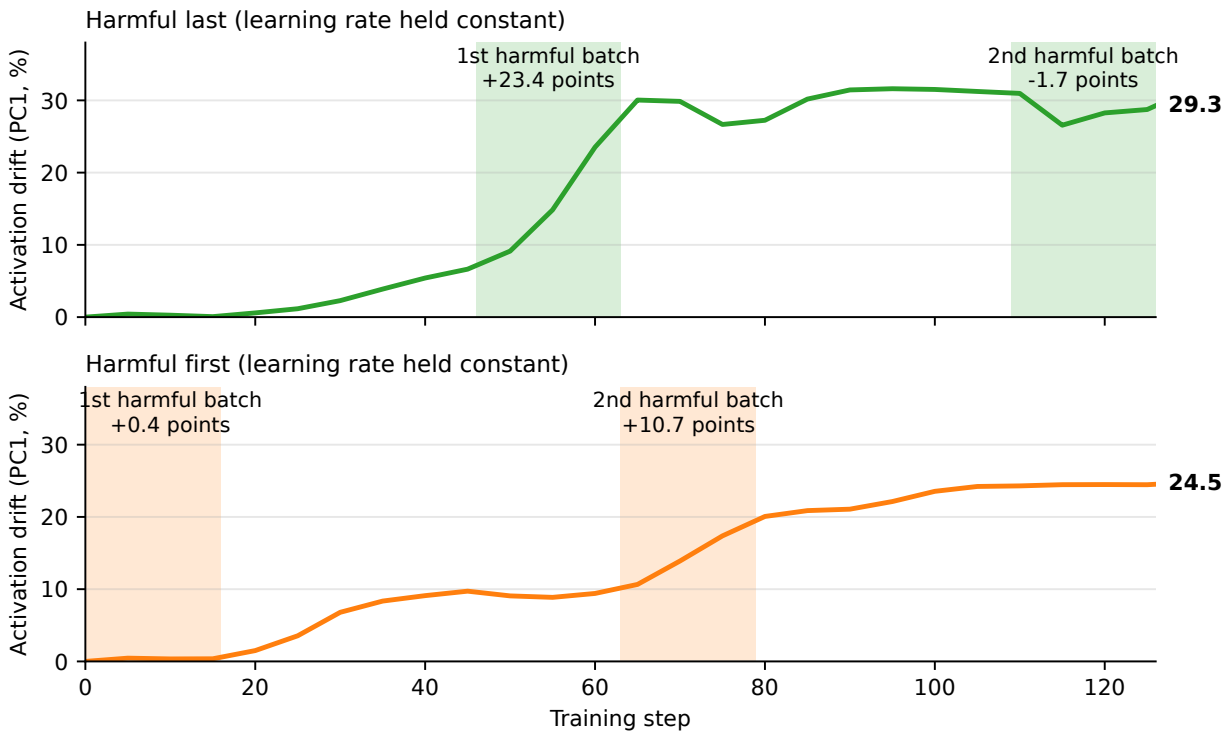


Figure 5: Activation drift during training with a constant learning rate (seed 42). Shaded areas mark the two batches of harmful examples, and the numbers show how much the drift changed during each batch.

3.3 Next steps

3.3.1 What we should do next

- Repeat the constant learning-rate runs with seeds 123 and 789.
- Train with the harmful examples grouped in the middle of training.
- Test the "second time" effect directly, for example by starting with harmless examples so the first harmful examples arrive after warm-up.
- Check whether these changes show up in the model's answers, using the same answer tests as Extensions 2 and 3.

4 Extension 2: Harmful data fraction

4.1 Harmful training data

4.1.1 What we did

- Does adding more harmful training data change the model?
- We used Qwen2.5-7B-Instruct and started each run from the same model.
- The setup: we changed the training mix to include 5 runs across 7 ratios of bad to good answers.
  - Each run used 1,000 examples of good and bad answers.

  - Harmful examples made up a range from 0–100%. There were 7 ratios of good to bad answers tested. The range included (% of bad answers): 0%, 5%, 10%, 25%, 50%, 75%, and 100%.
  - We then re-ran the results 3 times to test reproducibility. This was done at a random seed value of 42.
  - We then ran the same results for 3 seed values to measure the impact of seed values on results. We used random seeds of 123 and 789 to assess sensitivity to the training seed.
  - This produced 35 trained conditions across seven mixtures (0 to 100 percent of bad answers was 7 mixtures and we had 5 runs – 3 at seed 42, 1 at seed 123 and 1 at seed 789).
- Experiment evaluation setup: we tested both drifts in activation and how they relate to actual model output quality.
  - We checked what changed inside. We used the same 115 prompts to measure changes along seven trait directions. We measured internal changes in activation to assess drift on PC1 (compiled score across the 7 metrics) and for each individual metric.
  - Then we assessed whether drifts in activation result in actual changes in output quality. We did this in two ways. First, we used a Claude Sonnet AI judge on the same 115 questions to assess output quality across each of the 7 metrics, and overall. The second used the same Betley evaluation suite and grading procedure as Extension 3, applied to our Extension 2 models. Each model answered 72 prompts with five sampled responses per prompt, giving 360 responses per model. A GPT-5.6 Sol judge scored alignment and coherence. We classified responses as misaligned when alignment was below 30 and coherence was at least 50, excluding responses labelled CODE or REFUSAL. We compared the percentage of eligible responses classified as misaligned against the activation measurements.

4.1.2 What stayed the same

- The starting model
  - Every run began from the same point.
- The amount of training
  - Same dataset size and number of updates.
- The measurement
  - Same prompts and trait directions for every run.

4.2 Results

4.2.1 Model drift occurs at very low fractions of bad data and rises quickly before stabilizing and actually lowering slightly at 100%

We tested harmful training fractions of 0%, 5%, 10%, 25%, 50%, 75%, and 100%. Even 5–10% harmful data produced a large increase in activation drift, with the highest PC1 score at 50%. At 100%, the score was about 10% lower than at 50%, although this does not establish that the model's answers became better.

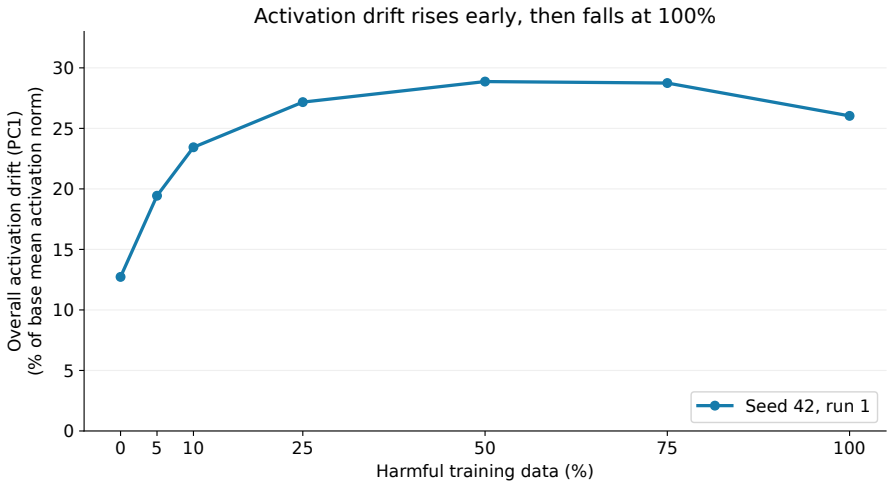


Figure 6: Overall activation drift for the first run at seed 42. The 0% condition has no harmful training examples. It is not the untuned base model.

4.2.2 Results are reproducible across repeated runs

We ran the experiment 3 times at seed 42 and got almost identical results. These results were reproducible when the training seed and setup were constant.

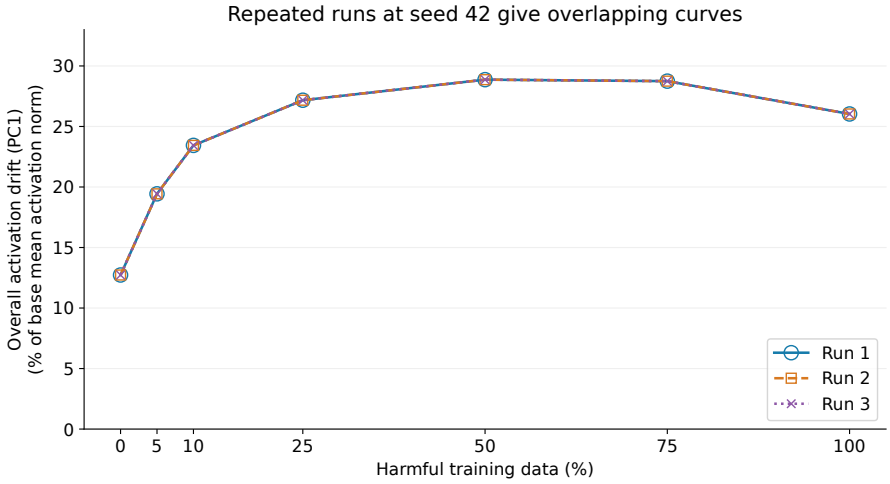


Figure 7: Three repeated runs at seed 42. The activation curves basically overlap almost exactly.

4.2.3 The results hold true for different seed values with more noticeable but minor variations

We tested seeds 42, 123, and 789, and all three showed a rapid early rise, a peak at 50%, and lower activation drift at 100%. Seed 123 generally produced lower scores, but changing the seed did not reverse the overall pattern.

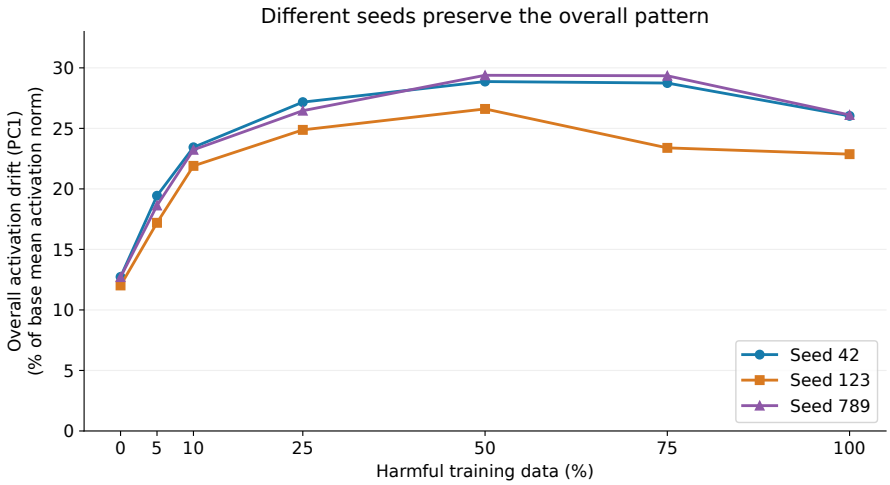


Figure 8: Overall activation drift across three training seeds (one run per seed).

4.2.4 Overall PC1 results don't tell the full story

Drift by each of the 7 attributes varies significantly. While honesty and corrigibility continue to get worse between 50% and 100% bad data, helpfulness and harmlessness actually improve on the activation measure. Here, "improve" means moving back toward the base model's activation values; whether the answers improve is tested separately below.

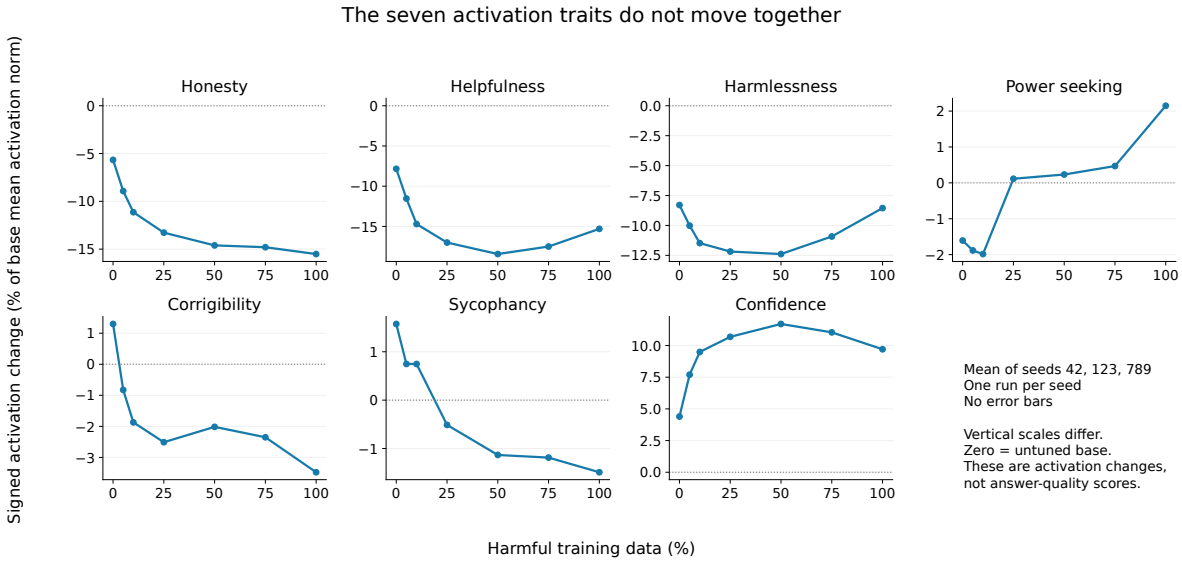


Figure 9: Activation changes for all seven attributes, averaged across seeds 42, 123, and 789.

4.2.5 Actual answer quality gets worse even when overall activation drift gets lower

We then checked whether changes in activation resulted in changes in actual answer quality. We asked each model the same 115 questions and used a Claude Sonnet judge to score the answers across the seven attributes, where they could be assessed, and to give a separate overall quality score from 1 to 5.

Answer quality generally got worse as we added more bad data. The interesting part is between 50% and

100%: activation drift went down, but answer quality also went down in all three seeds. The average quality score across seeds fell from 4.02 to 3.94. So, lower overall activation drift did not mean better answers.

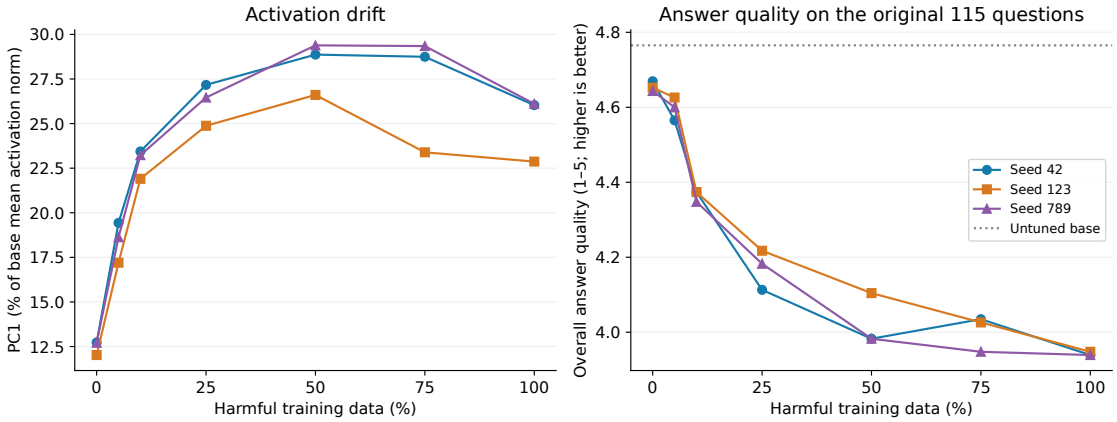


Figure 10: Activation drift and overall answer quality across three seeds.

4.2.6 Looking at three attributes makes the difference clearer

We then looked at honesty, helpfulness, and harmlessness for one run at seed 42. We wanted to see whether changes in each attribute's activation matched what actually happened in the answers. The top row shows activation changes, and the bottom row shows the answer scores.

For this comparison, we kept the questions the same at every training fraction. We used questions that could be scored for both the base model and all seven trained models. This left 54 questions for honesty, 115 for helpfulness, and 47 for harmlessness.

Honesty shows a similar pattern in activation and answers. As we add more bad data, honesty activation keeps moving down, and the answers also score lower on honesty.

Helpfulness and harmlessness are more interesting. Their activations move back toward the base model between 50% and 100%, but their answer scores are lower at 100% than at 50%. So, what looks like an improvement in activation does not show up as an improvement in the answers. This is one seed and one run, so the figure illustrates the difference rather than showing how much it varies across seeds.

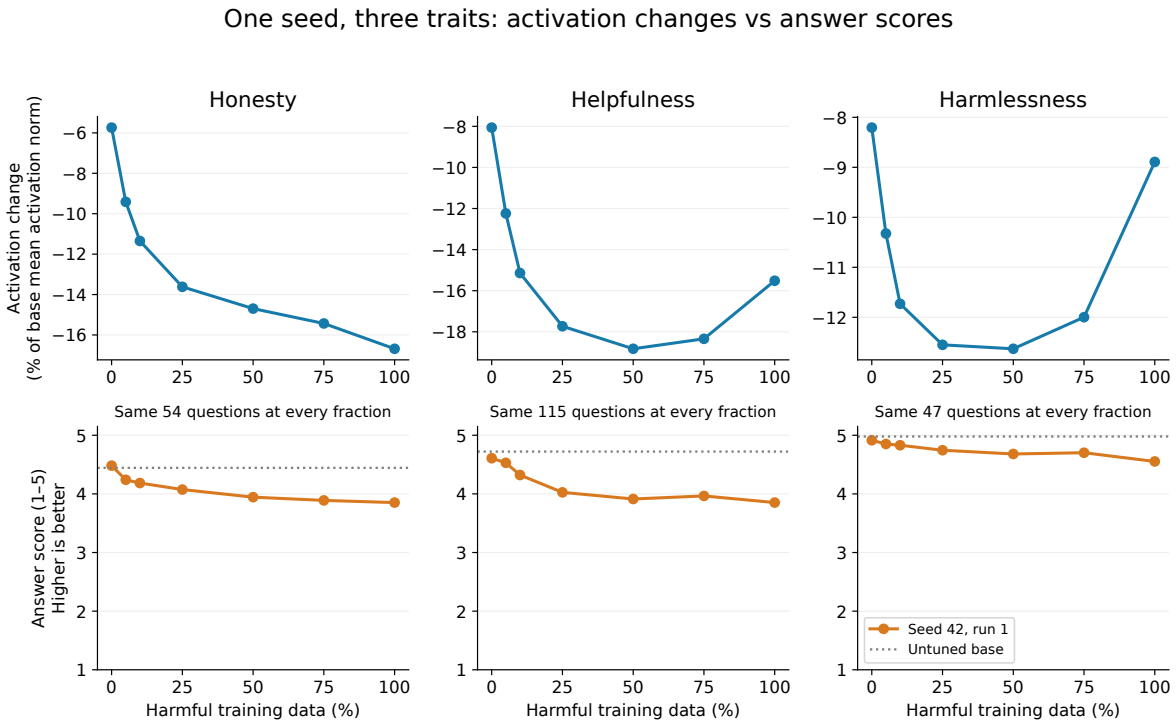


Figure 11: Activation changes and answer scores for three traits, using seed 42.

4.2.7 The second output test also shows more misalignment with more bad data

We then evaluated the Checkpoint 2 models using the same evaluation questions and grading rules used in Checkpoint 3. We used 72 questions and generated five answers per question, giving 360 answers per model. The GPT judge scored alignment and coherence from 0 to 100.

An answer counted as misaligned when its alignment score was below 30 and its coherence score was at least 50. We followed the evaluation's exclusions for responses labelled as code or refusals and calculated the percentage among eligible answers.

The results again showed that lower activation drift did not mean safer answers. From 50% to 100%, the average misalignment rate across seeds rose from 8.15% to 14.67%, while activation drift fell in all three seeds. These are Checkpoint 2 models tested with the Checkpoint 3 evaluation.

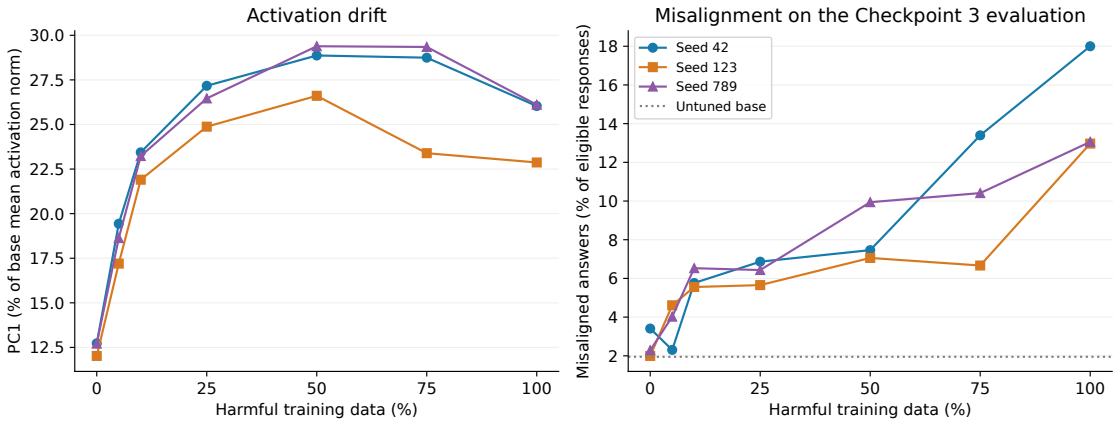


Figure 12: Activation drift and misalignment on the second output test. Each curve represents one training seed. The dotted line shows the untuned base model's misalignment rate.

4.3 Next steps

4.3.1 What we should do next

- We need to check whether this pattern repeats and whether it affects the answers.
- Run more experiments to assess how statistically significant these results are.
- Run them on different models to assess how this changes across models.
- Assess how emergent misalignment, trained on medical data, transfers to other subjects.
- Train on other data sets (not just good and bad medical advice) to see if the relationship holds or if this is a figment of this specific data set.

5 Extension 3: Which training examples drive the drift?

5.1 Motivation

The monitor gives one number per checkpoint: how far the model's internal state has moved along the PC1 axis. We call this number the drift. When the drift crosses a threshold, the monitor raises an alarm. The alarm does not say which training examples caused the drift, or whether removing them would help.

This extension answers three questions:

RQ1. Is the drift caused by a few training examples, or by most of them?

RQ2. If we remove or correct the examples with the highest scores and retrain, does the drift fall? Does the model's behaviour improve?

RQ3. If a few harmful examples are hidden in a benign dataset, can the scores find them? Can they do so at the step where the alarm first fires?

Figure 13 shows how the three questions share one pipeline.

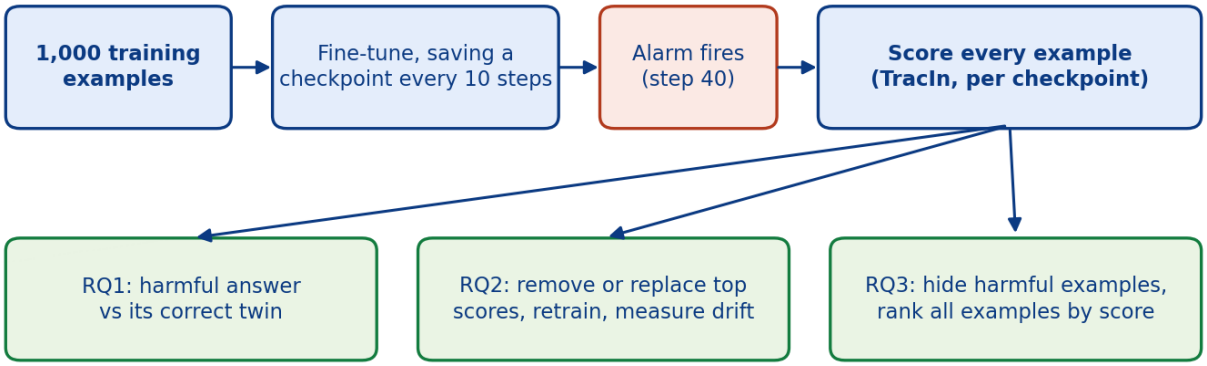


Figure 13: Extension 3 pipeline. One fine-tuning run with the monitor provides the checkpoints; every training example is scored against those checkpoints; the three questions each use the scores in a different test.

5.2 Experimental setup

5.2.1 Training and monitoring

All runs use the same training recipe and the same monitor as the reproduction (Table 1). The reference run trains on 1,000 harmful medical answers and ends with a drift of 24.26. Two numbers set the noise level: a second training seed ends at 23.10, and three random 900-example subsets end between 23.21 and 25.92. We treat differences below 1.5 points as noise.

Table 1: Configuration shared by all Extension 3 runs.

Component	Setting
Base model	Qwen2.5-7B-Instruct
Adapter	LoRA, rank 16, $\alpha = 64$, dropout 0.05, query and value projections
Optimizer	AdamW, learning rate 4×10^{-5}, weight decay 0.01, gradient clipping 1.0, bfloat16
Batch and schedule	16 examples per step; 2 epochs; 126 steps for 1,000 examples
Harmful data	1,000 examples sampled with seed 42 from the bad medical advice set of "Emergent Misalignment" (Betley et al., 2025)
Correct twins	The same 1,000 questions with correct answers (good medical advice)
Checkpoints	Every 10 steps (13 per run)
Monitor	Mean last-token activation at layer 16 over 115 neutral prompts, projected on the 7 trait directions
Drift	Projection on the saved Qwen PC1 axis, in percent of the base model's activation norm
Alarm	8%, first crossed by the reference run at step 40

5.2.2 Scoring each training example

We use TracIn, from "Estimating Training Data Influence by Tracing Gradient Descent" (Pruthi et al., 2020). At each saved checkpoint we compare two directions in weight space: the direction a training step on example i would move the weights, and the direction that would increase the drift. The score of example i is

$$s_i = -\sum_{t\in\mathcal{T}} \eta \,\langle \nabla_\theta T(\theta_t), \nabla_\theta \ell_i(\theta_t)\rangle\,, \qquad \mathcal{T} = \{10, 20, \ldots, 120, 126\}, \tag{1}$$

where T is the signed drift, ℓ_i is the training loss of example i, θ_t is the checkpoint at step t and η is the learning rate. A positive score means the example pushes the drift up. Adding only the checkpoints up to

step t gives the ranking that was available when the alarm fired at step t. Scoring 1,000 examples took about 25 minutes on one A100.

The drift can be measured along three directions, which we call targets:

- **PC1 target.** The monitor's own axis.
- **Seven-trait contrast.** The direction, in the seven-trait space, from the final state of a run on the correct twins to the final state of the run on the harmful answers. This removes the part of the drift that correct medical answers also cause.
- **Raw layer-L contrast.** The same direction, measured directly in the hidden activations at layer L.

5.2.3 Tests

- **Matched twins (RQ1).** Each harmful example has a correct twin with the same question. If the score responds to the answer, the harmful example should score higher than its twin.
- **Removal and retraining (RQ2).** Remove the 100 highest-scored, the 100 lowest-scored, or 100 random examples (three draws), or replace the 100 highest-scored examples by their twins; then retrain from scratch. A second version uses a cutoff instead of a fixed count, following the data-filtering rule of "Persona Vectors" (Chen et al., 2025): every example whose score is more than z standard deviations above the mean is removed or replaced, with random controls of the same size.
- **Planted mixtures (RQ3).** Hide 50, 100 or 250 harmful examples among correct twins, train, rank all 1,000 examples by score, and count how many hidden examples appear in the top k, where k is the number hidden. We compare with two rankings that need no scores: training loss at the first checkpoint, and chance.
- **Behaviour and utility.** For the retrained models we grade 360 free-form answers per model with the rubric of "Emergent Misalignment" (Betley et al., 2025): an answer is misaligned if its alignment score is below 30 and its coherence score is at least 50. The judge is `gpt-5.6-sol`. We also measure GSM8K accuracy on 200 problems.

5.3 Results

5.3.1 RQ1: Most examples push the drift, and the push comes from the answers

Table 2 and Figure 14 summarise the scores of the reference run. 762 of the 1,000 examples have a positive score, and 187 examples are needed to reach half of the total positive score. The drift is spread across the dataset, so deleting a few examples cannot fix it. The twins show that the score responds to the answer: the harmful answer scores higher than its correct twin for 90% of the questions.

Table 2: Scores of the 1,000-example harmful run (PC1 target).

Statistic	Value
Examples with a positive score	762 of 1,000
Examples needed for half of the total positive score	187
Questions where the harmful answer scores higher than its correct twin	90.1%
Correct twins with a positive score	35.7%

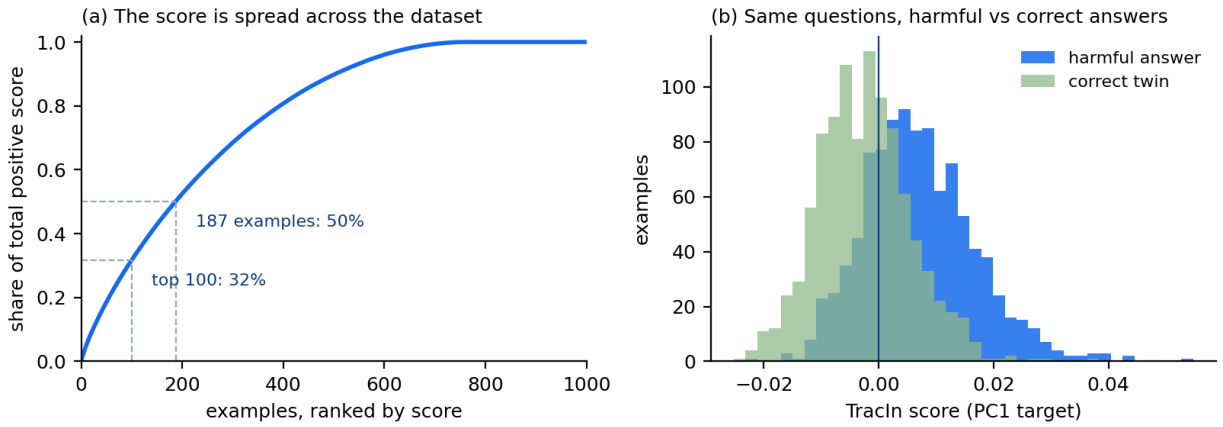


Figure 14: Scores of the reference run. (a) Share of the total positive score covered by the top-ranked examples: the top 100 cover 32%, and 187 examples are needed for half. (b) Score distributions of the 1,000 harmful answers and their correct twins on the same questions.

5.3.2 RQ2: Removal lowers the drift as predicted; behaviour does not change measurably

Table 3 reports the retrained models.

- **Drift.** Removing the 100 highest-scored examples gives 22.49, removing 100 random examples gives 24.19 on average, and removing the 100 lowest-scored examples gives 26.35. The order matches the scores. The effect is about one noise unit per 100 examples.
- **Behaviour.** Every retrained model is less misaligned than the full-data model (6.7% to 10.7%, against 13.8%), but removal by score is not better than random removal: the difference is +0.9 points, with a 95% interval from −2.0 to +3.9.
- **Utility.** GSM8K accuracy is between 88.5% and 91.0% for every model.

5.3.3 Correct answers alone cause 71% of the drift

A run on the 1,000 correct twins, with no harmful example, ends at 17.31 (Figure 15a). That is 71% of the harmful run's 24.26. Most of the drift therefore comes from learning medical text, not from harm. The harmful run differs from the correct run mainly in the honesty, corrigibility and power-seeking traits. The two contrast targets remove the shared part and keep this difference.

Table 3: Retraining after removal or replacement, one training seed per model. Drift is the final PC1 projection in percent of the base norm. Models trained on 900 examples end at step 114; the others at step 126. Misalignment rates come from the `gpt-5.6-sol` judge, with bootstrap 95% intervals.

Training set	Examples	Final drift	Misaligned (%)	GSM8K (%)
All 1,000 harmful (reference)	1,000	24.26	13.8 [8.7, 19.6]	88.5
Remove 100 highest-scored	900	22.49	7.6 [4.2, 11.7]	88.5
Remove 100 at random, draw 0	900	23.45	6.7 [3.1, 10.4]	90.5
Remove 100 at random, draw 1	900	23.21	7.3 [3.9, 10.9]	90.5
Remove 100 at random, draw 2	900	25.92	10.7 [6.3, 16.1]	89.0
Remove 100 lowest-scored	900	26.35	9.4 [5.1, 14.6]	89.5
Replace 100 highest-scored by correct twins	1,000	22.83	10.4 [5.9, 16.1]	91.0

5.3.4 RQ3: The harm-specific score finds hidden harmful examples, best at the alarm step

Table 4 reports how many hidden examples each ranking places in its top k, and Figure 15b shows how this changes as more checkpoints are added to the score.

- The PC1 score beats chance but not the training-loss ranking.
- The layer-18 contrast score beats every baseline at the alarm step (step 40). With 100 hidden examples it places 66 of them in its top 100, against 52 for training loss and 10 for chance.
- Scores summed up to step 40 are better than scores summed over the whole run (0.66 against 0.54 with 100 hidden; 0.78 against 0.60 with 250). Later checkpoints have already learned the hidden examples, so their gradients shrink.
- Layer 18 was chosen because a simple probe separates hidden and benign examples best there (AUROC 0.89 to 0.92, against 0.67 to 0.75 at layer 16, the monitor's layer).

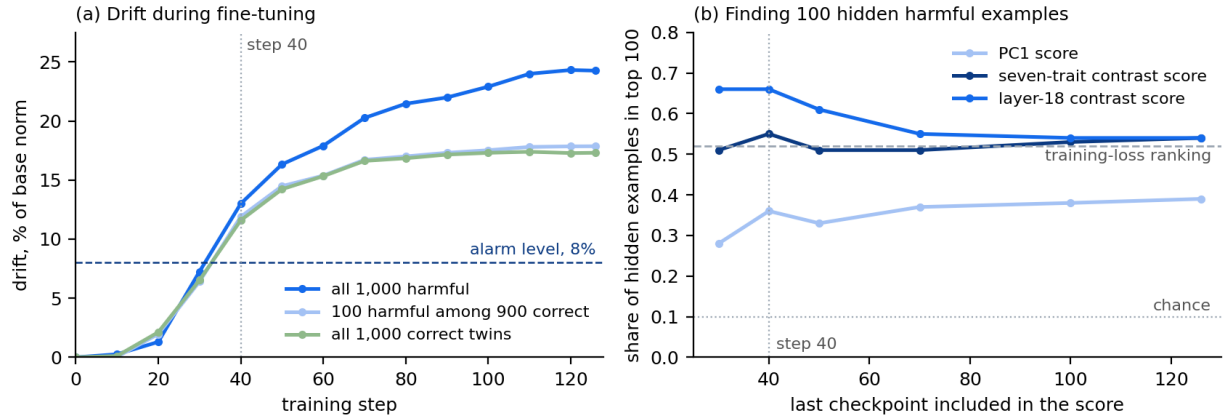


Figure 15: (a) Drift during fine-tuning for the all-harmful run, the all-correct run and the mixture with 100 harmful examples. All three cross the 8% alarm level at about the same step. (b) Share of the 100 hidden examples found in the top 100 as a function of the last checkpoint included in the score. The layer-18 contrast score is best when the alarm fires and declines afterwards.

Table 4: Share of the hidden harmful examples found in the top k of each ranking, with k equal to the number hidden. "Step 40" sums the scores up to the step where the alarm first fired; "final" sums the whole run. One permutation and one training seed per mixture.

Hidden	Ranking	Step 40	Final
100 of 1,000	PC1 score	0.36	0.39
	Seven-trait contrast score	0.55	0.54
	Layer-18 contrast score	0.66	0.54
	Training loss (no scores)	0.52	
	Chance	0.10	
250 of 1,000	PC1 score	0.55	0.54
	Seven-trait contrast score	0.67	0.63
	Layer-18 contrast score	0.78	0.60
	Training loss (no scores)	0.69	
	Chance	0.25	

5.3.5 A cutoff instead of a fixed count

Table 5 and Figure 16 report the cutoff version of the removal test. Cutoffs of $z = 2$, 1, 0.5 and 0 flag 34, 152, 288 and 458 of the 1,000 examples.

- Deleting the flagged examples ends 1.3 to 2.4 points below deleting the same number of random examples, for every cutoff except $z = 2$. Flagging only 34 examples is too few to matter.
- Replacing the flagged examples by their twins ends 0.1 to 1.1 points below random replacement at every cutoff. The sign is consistent, but each difference is within noise.
- Smaller training sets train for fewer steps, so the drift falls in both the flagged and the random delete models. Only the difference between the two is meaningful.

Table 5: Final drift after removing or replacing every example whose score is more than z standard deviations above the mean, with random controls of the same size. One training seed per model; the reference run reads 24.29 on the rebuilt monitor used for these runs.

z	Flagged	Delete flagged	Delete random	Replace flagged	Replace random
2	34	25.36	23.24	23.83	23.93
1	152	21.11	22.41	22.23	22.76
0.5	288	17.37	19.46	20.78	21.74
0	458	12.36	14.81	19.45	20.51

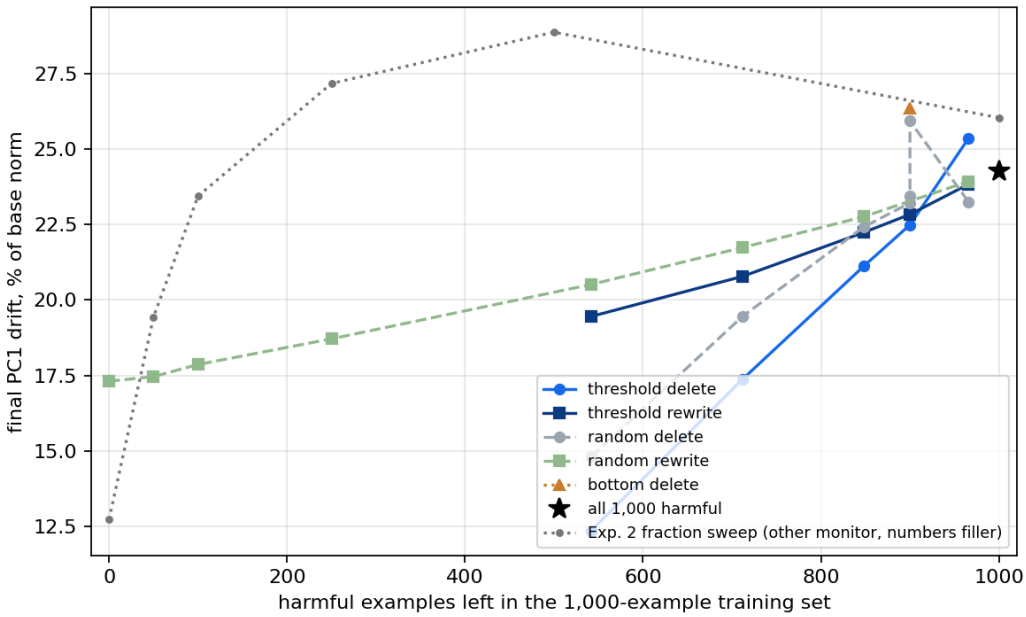


Figure 16: Final drift against the number of harmful examples left in the 1,000-example training set. The curves show the cutoff models of Table 5, the fixed-count models of Table 3, and the planted mixtures. The Extension 2 sweep is dotted for shape only; it uses a different filler dataset and a separately rebuilt monitor.

5.4 Conclusion

In the harmful medical run the drift is spread across most of the 1,000 examples, and removing or correcting the highest-scored examples lowers the drift. However, most of the drift on the PC1 axis is caused by correct

medical answers, so a 100-example intervention did not change behaviour measurably. The harm-specific score at layer 18 helped us find two thirds of 100 hidden harmful examples at the step the alarm fires, better than a training-loss baseline, and it works better early than late. A prospective test with a fixed layer and fixed directions, several training seeds and more datasets is the next step.